5) 4x4 sudoku:

```
states = ["TN", "KL", "KA", "AP", "TG", "GA", "MH"]
colors = ["Red", "Green", "Blue"]

neighbors = {
    "TN": ["KL", "KA", "AP"],
    "KL": ["TN", "KA"],
    "KA": ["TN", "KL", "AP", "TG", "GA", "MH"],
    "AP": ["TN", "KA", "TG"],
    "TG": ["KA", "AP", "MH"],
    "GA": ["KA", "MH"],
    "MH": ["KA", "GA", "TG"]
}

domains = {s: colors[:] for s in states}
assignment = {}

def select_state():
    unassigned = [s for s in states if s not in assignment]

    # Degree heuristic: choose first variable
    if len(assignment) == 0:
        return max(unassigned, key=lambda s: len(neighbors[s]))

    # MRV, with Degree heuristic as tie-breaker
    return min(
        unassigned,
        key=lambda s: (
            len(domains[s]),
            -sum(n not in assignment for n in neighbors[s])
        )
    )

def backtrack():
    if len(assignment) == len(states):
        return True

    state = select_state()
    print("\nSelected state:", state)

    for color in domains[state][:]:
        # Check neighbouring constraints
```

```
        if any(assignment.get(n) == color
               for n in neighbors[state]):
            continue

        old_domains = {s: d[:] for s, d in domains.items()}

        assignment[state] = color
        domains[state] = [color]

        valid = True

        # Forward checking
        for n in neighbors[state]:
            if n not in assignment and color in domains[n]:
                domains[n].remove(color)

                if not domains[n]:
                    valid = False

        print("Assigned:", state, "=", color)
        print("Domains:", domains)

        if valid and backtrack():
            return True

        # Backtracking: restore previous domains
        del assignment[state]
        for s in states:
            domains[s] = old_domains[s]

        print("Backtracking from", state)

    return False

if backtrack():
    print("\nFinal Colouring:")
    for state in states:
        print(state, "=", assignment[state])

    print("\nChecking constraints:")
    for state in states:
        for n in neighbors[state]:
            assert assignment[state] != assignment[n]
```

```
    print("All neighbouring states have different colours!")
else:
    print("No solution exists.")
```